

Egypt Travel Guide RAG QA System Report

1. Dataset Description

The selected website is Egypt Travel Guide : <https://www.egypttravelguide.com/>.

The site was selected because it contains structured travel-related information about Egypt, including tours, Red Sea day trips, airport transfers, destination pages, blog articles, prices, and short tour descriptions.

Main source URLs:

- <https://www.egypttravelguide.com/>
- <https://www.egypttravelguide.com/hurghada>
- <https://www.egypttravelguide.com/marsa-alam>
- <https://www.egypttravelguide.com/sharm-el-sheikh>
- <https://www.egypttravelguide.com/transfer>
- <https://www.egypttravelguide.com/blog>
- Tour pages discovered from internal links containing `/tour/`

The collected data type is public HTML pages converted into clean JSONL records.

Expected final dataset format:

```
{
  "url": "source page URL",
  "title": "page title",
  "page_type": "tour / blog / transfer / destination / package / general",
  "price_eur": 45.0,
  "locations": ["Hurghada"],
  "activities": ["snorkeling", "family"],
  "sections": {
    "overview": "clean text"
  },
  "full_text": "combined cleaned page text"
}
```

2. Data Collection Method

Data is collected using Python:

- `requests` for downloading public HTML pages
- `BeautifulSoup` for parsing HTML
- `urllib.robotparser` for checking robots.txt when available
- `tqdm` for progress display

The scraper starts from seed pages such as the homepage, destination pages, transfer page, blog page, and package tour page. It then discovers internal public links and keeps useful URLs such as tour pages, blog pages, destination pages, and transfer pages.

The scraper is intentionally polite:

- It uses a user-agent identifying the project as educational.
- It includes a delay between requests.

- It limits the maximum number of pages.
- It avoids checkout, account, image, PDF, and static asset URLs.

Cleaning steps:

- Remove scripts, styles, SVGs, and noscript tags.
- Extract headings, paragraphs, list items, and table cells.
- Normalize whitespace.
- Remove navigation and social media noise where possible.
- Deduplicate pages by URL.
- Keep only pages with meaningful text length.

Challenges:

- Some pages contain repeated navigation/menu text.
- Tour cards on listing pages contain compact text and may not have all details.
- Some metadata such as duration may not always be visible in a consistent structure.
- Prices and locations are extracted with heuristics, so they may require manual review.

3. Chunking Strategy

The chosen strategy is section-aware chunking .

Each page is first divided by natural HTML headings such as `h1`, `h2`, `h3`, and `h4`. Each section becomes one chunk if it is short enough. If a section is too long, it is split into overlapping text windows.

Chosen parameters:

- Maximum chunk size: 1200 characters
- Overlap: 150 characters
- Metadata stored for every chunk:
- source URL
- page title
- page type
- section title
- price in EUR if detected
- locations
- activities

Why this strategy was chosen:

Travel pages usually have meaningful sections, such as overview, included items, warnings, itinerary, and notes. Splitting by section preserves context better than arbitrary sentence splitting.

Alternatives considered:

1. Full page as one chunk
Rejected because long pages may contain several unrelated topics and retrieval would be less precise.
2. Fixed-size chunks only
Rejected because it may split an itinerary or warning section in the wrong place.

3. Sentence-level chunking

Rejected because single sentences may lose important context such as tour title, location, and price.

Weaknesses:

- Some sections may be very short.
- Some pages may not use clear headings.
- Section titles may include noisy text.

Mitigation:

- Very long sections are split with overlap.
- Metadata is injected into every chunk.
- Short/noisy sections are filtered during cleaning.

4. Embedding and Vector Store

Embedding model: `BAAI/bge-small-en-v1.5`

Reason for selection:

- Good English semantic retrieval performance for small projects.
- Free and local.
- Fast enough for a student laptop.
- Small embedding size compared with larger models.
- Suitable for a moderate number of travel-guide chunks.

Vector store: FAISS

Reason for selection:

- Lightweight local vector index.
- Easy to save and load.
- Good for educational RAG systems.
- Does not require a server.

The FAISS index uses normalized embeddings and inner product similarity, which behaves like cosine similarity when vectors are normalized.

5. Gemini API Usage

Gemini is used in two places:

1. Optional query intelligence

2. Rewrite user query into clear English.

3. Classify the query.

4. Extract structured filters.
5. Final answer generation
6. Retrieved chunks are passed as context.
7. Gemini generates an answer grounded only in that context.

Grounding prompt strategy:

The prompt tells Gemini:

- Answer only using the retrieved context.
- Do not invent prices, locations, times, or availability.
- Say that the answer was not found if the context is insufficient.
- Mention source titles used.

This reduces hallucination because Gemini is not asked to answer from general knowledge.

6. Query Intelligence

The query intelligence layer follows this structure:

Original query → Query rewriting → Query classification → Filter extraction → Filtered retrieval → Answer generation

Query classes used:

- factual_lookup
- recommendation
- comparison
- price_filter
- location_query
- transfer_query
- out_of_scope

Examples:

User query	Query class	Extracted filters
What is included in Orange Bay Island?	factual_lookup	activity = boat_trip
Recommend a trip for children in Hurghada	recommendation	location = Hurghada, activity = family
Compare Sharm El Naga and Orange Bay	comparison	activity = snorkeling
Snorkeling tours under 40 EUR	price_filter	max_price_eur = 40, activity = snorkeling
Tours from Marsa Alam	location_query	location = Marsa Alam
Hurghada airport transfer price	transfer_query	page_type = transfer, location = Hurghada
Who won the World Cup?	out_of_scope	none

The extracted filters are represented as a Pydantic object and applied to metadata where possible.

7. Hybrid Search

Hybrid search combines:

- Dense retrieval using embeddings + FAISS
- Sparse retrieval using BM25 keyword search

Dense retrieval is useful because it understands meaning. For example, it can connect “family friendly sea activity” with a tour that mentions children and Red Sea activities.

Sparse retrieval is useful because it catches exact words such as:

- Sindbad
- Orange Bay
- Sharm El Naga
- EUR 30
- airport transfer

Fusion method: Reciprocal Rank Fusion

Formula:

```
score = dense_weight * 1 / (rrf_k + dense_rank)
        + sparse_weight * 1 / (rrf_k + sparse_rank)
```

Default weights:

- Dense: 0.55
- Sparse: 0.45
- `rrf_k`: 60

Dense-only vs hybrid comparison should be generated by running:

```
python -m src.evaluate_retrieval --output evaluation/dense_vs_hybrid.md
```

Required evaluation queries:

1. Is Sindbad Submarine suitable for claustrophobic people?
2. Cheap snorkeling tours from Hurghada under 40 EUR
3. Compare Sharm El Naga and Orange Bay

After running the system, screenshots or copied outputs should be added to the final report.

8. Streamlit GUI

The Streamlit GUI is implemented in `app.py`.

It contains:

- Text input box for the user question
- Submit button
- Retrieval mode selector: dense, sparse, or hybrid
- Top-k slider

- Final generated answer
- Original query
- Rewritten query
- Query class
- Extracted filters
- Retrieved chunks
- Metadata
- Scores

The GUI connects to the core pipeline through `EgyptTravelRAGPipeline` in `src/pipeline.py`.

Screenshots to add before submission:

- App homepage
- Example answer
- Retrieved chunks expander
- Query intelligence section

9. Limitations

Current limitations:

- The scraper depends on the current website HTML structure.
- Some pages may not have clean section headings.
- Some metadata is inferred using keyword rules.
- Price extraction may miss complex prices or multiple prices on a page.
- The dataset size depends on the crawl limit.
- Query filters are simple and may not cover all travel concepts.
- Gemini may still produce weak answers if retrieved chunks are weak.
- The system has no chat history.

10. Future Work

Possible improvements:

- Add better metadata extraction for duration, pickup location, included items, and exclusions.
- Add reranking after hybrid retrieval.
- Store data in Chroma or another persistent vector database.
- Add multilingual support for Arabic and German queries.
- Add chat history.
- Add a feedback button in Streamlit.
- Deploy the app on Streamlit Community Cloud.
- Add a more formal evaluation dataset with expected answers.